Distributed Systems

Spring 2018

International Institute of Information Technology

Hyderabad, India

Most slides taken from the textbook slides posted. Some slides from the lecture material by Sukumar Ghosh (U. Iowa) and Indranil Gupta (UIUC) and Pallab Dasgupta

Solution to Critical-Section Problem

1. Mutual Exclusion - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections
2. Progress - If no process is executing in its critical section and there exist some processes that wishes to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely
3. Bounded Waiting - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted
   - Assume that each process executes at a nonzero speed
   - No assumption concerning relative speed of the N processes

Mutual Exclusion

- Requirements of Mutual Exclusion Algorithms
  1. Safety Property: At any instant, only one process can execute the critical section.
  2. Liveness Property: Two or more sites should not endlessly wait for messages which will never arrive. A requesting site should get a chance to execute CS in finite time.
  3. Fairness: Each process gets a fair chance to execute the CS. Fairness property generally means the CS execution requests are executed in the order of their arrival (time is determined by a logical clock) in the system.

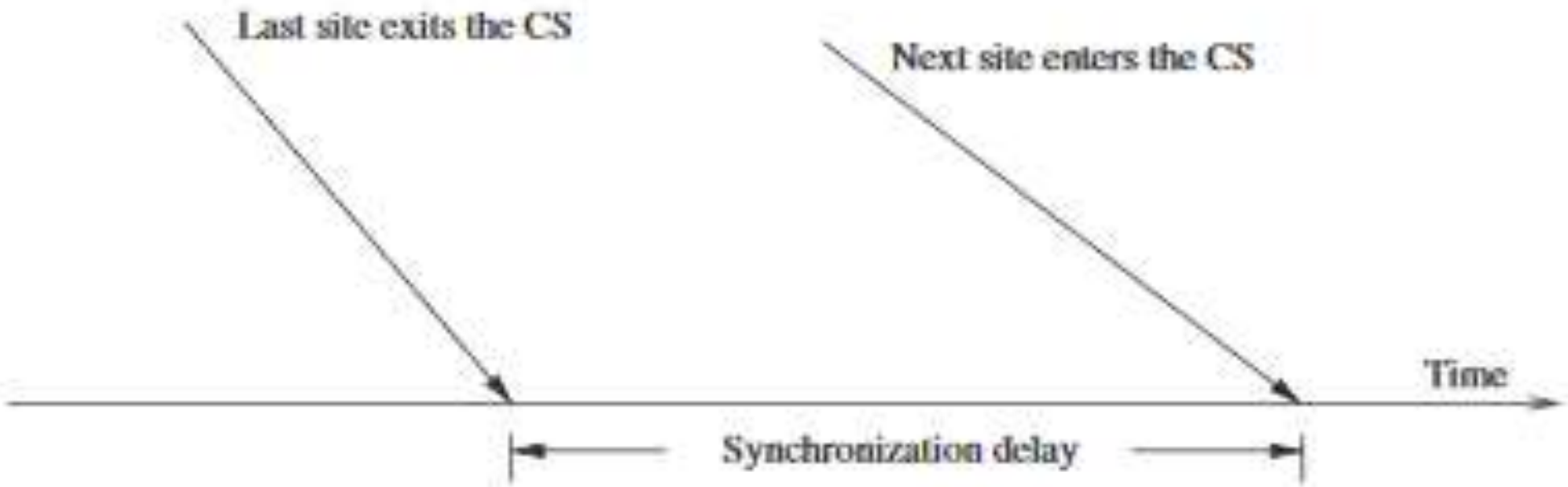
Performance Metrics

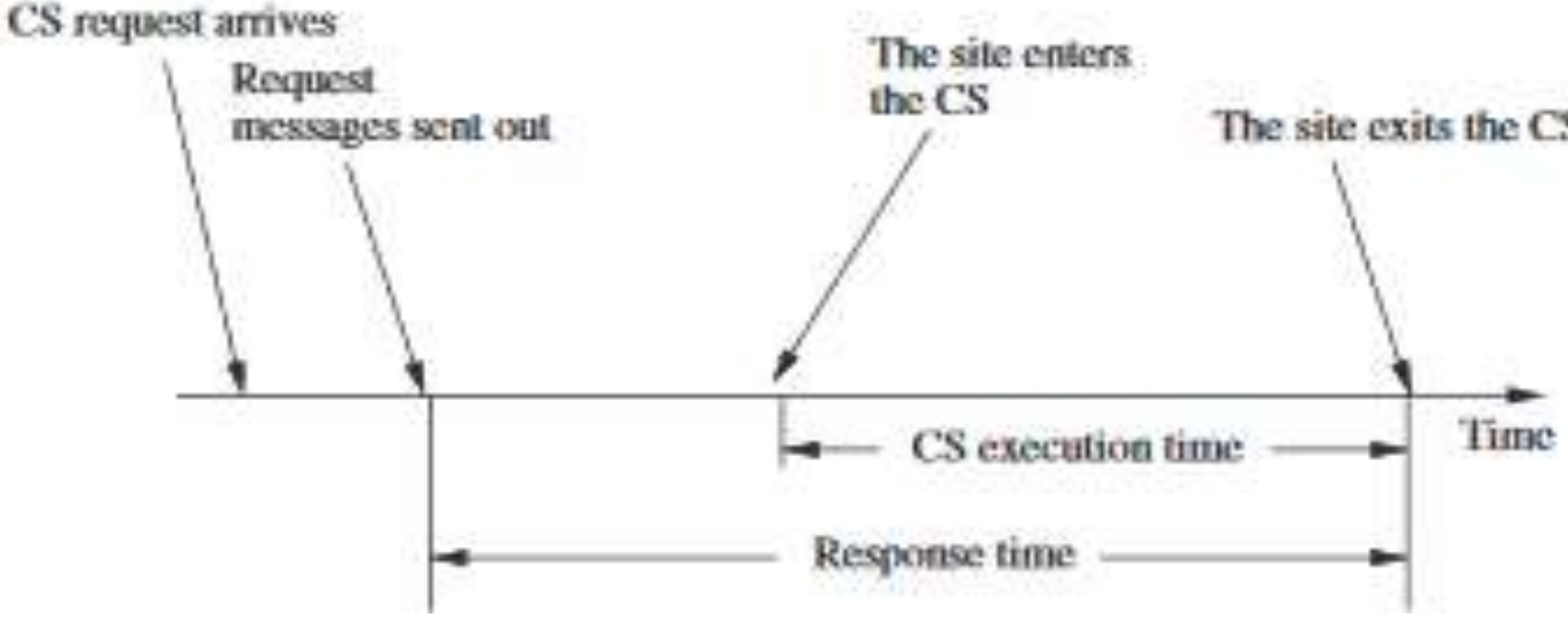
- Message complexity -number of messages that are required per CS execution by a site

- Synchronisation Delay - After a site leaves the CS, it is the time required before the next site enters CS

- Response Time - This is the time interval a request waits for its CS execution to be over after its request messages have been sent out .

- System Throughput - Rate at which the system executes requests for the CS.
  SD -synchronization delay E-avg CS execution time
  System throughput= 1/ (SD+E)


Last site exits the CS
Next site enters the CS
Time
Synchronization delay


CS request arrives
Request
messages sent out
The site enters
the CS
The site exits the CS
Time
CS execution time
Response time

- **Non-token based / Permission based**
  - Permission from all processes: e.g. Lamport, Ricart-Agarwala, Raicourol-Carvalho etc.
  - Permission from a subset: ex. Maekawa
- **Token based**
  - ex. Suzuki-Kasami

Lamport's Algorithm for Mutual Exclusion

- Lamport used his logical scalar clocks and FIFO assumption on message delivery to design an algorithm for mutual exclusion.
- Also assume a bidirectional channel between each pair of processors.
- Every site Si keeps a queue, $request\_queue_i$, which contains mutual exclusion requests ordered by their timestamps(priority queue). The request_queue is a set of waiting processes ordered by timestamp

Lamport's Algorithm for Mutual Exclusion

- Requesting the critical section:
  - When a site S_i wants to enter the CS, it broadcasts a REQUEST(ts_i , i) message to all other sites and places the request on request_queuei. ((ts_i , i) denotes the timestamp of the request.)
  - When a site S_k receives the REQUEST(ts_i , i) message from site S_i ,places site S_i 's request on $request\_queue_k$ and it returns a timestamped REPLY message to S_i.
- Executing the critical section:
  - Site S_i enters the CS when the following two conditions hold:
  - L1: S_i has received a message with timestamp larger than (ts_i, i) from all other sites. note: need not be reply msg
  - L2: S_i 's request is at the top of request_queuei.

Lamport's Algorithm

- Releasing the critical section:
  - Site S_i, upon exiting the CS, removes its request from the top of its request queue and broadcasts a timestamped RELEASE message to all other sites.
  - When a site S_j receives a RELEASE message from site S_i, it removes S_i's request from its request queue.
  - When a site removes a request from its request queue, its own request may come at the top of the queue, enabling it to enter the CS.

Lamport's Algorithm for Mutual Exclusion

Why do we need two conditions. Instead why cant Pi just enter CS when he is on the top of the queue ?

How does reply with higher timestamp take care of this?

How come it works for "any" message with larger timestamp. Need not even be a reply msg?

Scalar clock is not strongly consistent. Does this cause a problem?

Two timestamps of two processes could be the same. How can this be managed?

Can this work for non FIFO channels ?

Lamport's Algorithm

- Releasing the critical section:
  - Site S_i, upon exiting the CS, removes its request from the top of its request queue and broadcasts a timestamped RELEASE message to all other sites.
  - When a site S_j receives a RELEASE message from site S_i, it removes S_i's request from its request queue.
  - When a site removes a request from its request queue, its own request may come at the top of the queue, enabling it to enter the CS.

    Would this site have received a reply from the site that sent the RELEASE msg?

Lamport's Algorithm for Mutual Exclusion

Does it guarantee the ME conditions: (prove)

-Two persons do not enter at the same time

-Atleast one definitely enters if requesting processes exist

-Fairness

- **How many messages per critical section invocation**

- **How much is the Synchronization delay**

  Synchronisation Delay - After a site leaves the CS, it is the time required before the next site enters CS

Some notable points

- **Purpose of REPLY messages from node *i* to *j* is to ensure that *j* knows of all requests of *i* prior to sending the REPLY (and therefore, possibly any request of *i* with timestamp lower than *j*'s request)**
- **Requires FIFO channels.**
- **3(*n* – 1) messages per critical section invocation**
- **Synchronization delay = max mesg transmission time of Release msg**
- **Requests are granted in order of increasing timestamps**

Can we do away with the Release messages and manage only with request messages and reply msgs?

Optimisation -Ricart-Agarwala's Algorithm

Removes release msg.

If Process Pj has a request

- a process Pj gives reply msg only if the process has request of higher timestamp than received request
- If it has lower timestamp then give reply only after done with CS

If Process Pj has no request then give reply.

Reply message is a permission giving message

You enter CS only after receiving reply from all processes.

Ricart-Agarwala's Algorithm

- Uses two types of messages: REQUEST and REPLY.
- A process sends a REQUEST message to all other processes to request their permission to enter the critical section.
- A process sends a REPLY message to a process to give its permission to that process.
- Processes use Lamport-style logical clocks to assign a timestamp to critical section requests and timestamps are used to decide the priority of requests.
- Each process p_i maintains a Boolean Request-Deferred array, RD_i, the size of which is the same as the number of processes in the system.
- Initially, $\forall i \; \forall j$: $Rd_i[j]=0$. Whenever p_i defers the request sent by p_j, it sets $Rd_i[j]=1$ and after it has sent a REPLY message to p_j, it sets $RD_i[j]=0$.

Ricart-Agarwala's Algorithm

- Requesting the critical section:
  1. When a site S_i wants to enter the CS, it broadcasts a timestamped REQUEST message to all other sites.
  2. When site S_k receives a REQUEST message from site S_i, it sends a REPLY message to site S_i if site S_k is neither requesting nor executing the CS, or if the site S_k is requesting and S_i's request's timestamp is smaller than site S_k's own request's timestamp. Otherwise, the reply is deferred and S_j sets $RD_j[i]=1$.
- Executing the critical section:
  - Site S_i enters the CS after it has received a REPLY message from every site it sent a REQUEST message to.

Ricart-Agarwala's Algorithm

- Releasing the critical section:
  - When site S_i exits the CS, it sends all the deferred REPLY messages: ∀ j if $RD_i[j]=1$, then send a REPLY message to S_j and set $RD_i[j]=0$.

Ricart-Agarwala's Algorithm

- For each CS execution, Ricart-Agrawala algorithm requires (N − 1) REQUEST messages and (N − 1) REPLY messages.
- Thus, it requires 2(N − 1) messages per CS execution.

Ricart-Agarwala's Algorithm

- Do I need a heap ?

- Will it work for non FIFO?

The Ricart-Agrawala Algorithm

- Improvement over Lamport's
- Main Idea:
  - node *j* need not send a REPLY to node *i* if *j* has a request with timestamp lower than the request of *i* (since *i* cannot enter before *j* anyway in this case)
- Does not require FIFO
- 2(n – 1) messages per critical section invocation
- Synchronization delay = max. message transmission time
- Requests granted in order of increasing timestamps

- What if I am done with my CS and want to re-enter CS... Do I need to release CS and restart with sending requests or can I be smarter ?

Roucairol-Carvalho Algorithm

- **Improvement over Ricart-Agarwala**

- **Main idea**
  - **Once *i* has received a REPLY from *j*, it does not need to send a REQUEST to *j* again to re enter CS unless it has already sent a REPLY to *j* after the first CS(in response to a REQUEST from *j*)**
  - **Message complexity varies between 0 and 2(*n* – 1) depending on the request pattern**
  - **worst case message complexity still the same**

- **Non-token based / Permission based**
  - Permission from all processes: e.g. Lamport, Ricart-Agarwala, Raicourol-Carvalho etc.
  - Permission from a subset: ex. Maekawa
- **Token based**
  - ex. Suzuki-Kasami

Token based Algorithms

- **Single token circulates, enter CS when token is present**
- **Mutual exclusion obvious**
- **Algorithms differ in how to find and get the token**
- **Uses sequence numbers rather than timestamps to differentiate between old and current requests**

Suzuki Kasami Algorithm

- **Broadcast a request for the token**
- **Process with the token sends it to the requestor if it does not need it**
- **Issues:**
  - **Current versus outdated requests**
  - **Determining sites with pending requests**
  - **Deciding which site to give the token to**

Suzuki Kasami Algorithm

- The token has two parts in it:
  - Queue (FIFO) Q of requesting processes
  - LN[1..n] : where LN[j] is sequence number of request that *j* executed most recently (each time process j makes request for token to enter CS, the sequence number goes up. Next request by a process can be placed only after it gets token and enters CS. Hence LN[j] is the number of times it has already entered CS).

- The request message:
  - REQUEST(*i*, *k*): request message from node *i* for its k^{th} critical section execution

- Other data structures
  - RN_i[1..n] for each node *i*, where $RN_i[j]$ is the largest sequence number received so far by *i* in a REQUEST message from *j*.

- The token has two parts in it:
  - Queue (FIFO) Q of requesting processes
  - LN[1..n] : where LN[j] is sequence number of request that j executed most
- The request message:
  - REQUEST(i, k): request message from node i for its k^{th} critical section execution
- Other data structures
  - RN_i[1..n] for each node i, where $RN_i[\ j\ ]$ is the largest sequence number received so far by i in a REQUEST message from j.

What can be the gap in $RN_i[\ j\ ]$ and LN[j] ?

Suzuki Kasami Algorithm

- **To request critical section:**
  - **If *i* does not have token, increment $RN_i[\ i\ ]$ and send REQUEST(*i*, $RN_i[\ i\ ]$) to all nodes**
  - **If *i* has token already (means he got it to do a CS and now he is done with one CS but may not have released the token and wants to do the next CS), enter critical section if the token is idle (no pending requests), else follow rule to release critical section**

- **On receiving REQUEST(*i*, *sn*) at *j*:**
  - **Set $RN_j[\ i\ ]$ = max($RN_j[\ i\ ]$, *sn*)**
  - **If *j* has the token and the token is idle, then send it to *i* if $RN_j[\ i\ ]$ = LN[*i*] + 1. If token is not idle, follow rule to release critical section**

Suzuki Kasami Algorithm

- **To enter critical section:**
  - **Enter CS if token is present**
- **To release critical section:**
  - **Set LN[i] = RN_i[i]**
  - **For every node j which is not in Q (in token), add node j to Q if RN_i[j] = LN[j] + 1 (these are requests that arrived while CS was executing)**
  - **If Q is non empty after the above, delete first node from Q and send the token to that node**

Notable features

- **No. of messages:**
  - **0 if node holds the token already, n otherwise**
- **Synchronization delay:**
  - **0 (node has the token) or max. message delay (token is elsewhere)**
- **No starvation**

Raymond's Algorithm

- **Forms a directed k-ary tree (logical) with the token-holder as root which will keep reorienting itself**

- **Each node has variable "*Holder*" that points to its parent on the path to the root.**
  - **Root's Holder variable points to itself**

- **Each node i has a FIFO request queue Q_i**

Raymond's Algorithm

- **To request critical section:**
  - **place request in Q_i**
  - **If i does not hold the token currently and Q_i is empty then send REQUEST to parent on the tree.**
- **When a non-root node j receives a request from i**
  - **place request in Q_j**
  - **send REQUEST to parent if no previous REQUEST sent**

Raymond's Algorithm

- **When the root receives a REQUEST:**
  - **send the token to the requesting node**
  - **set *Holder* variable to point to that node**

- **When a node receives the token:**
  - **delete first entry from the queue**
  - **send token to that node**
  - **set *Holder* variable to point to that node**
  - **if queue is non-empty, send a REQUEST message to the parent (node pointed at by *Holder* variable) {because you want it to come back to serve the pending request}**

    requests in the queue of a process are requests by whom ?

Raymond's Algorithm

- **To execute critical section:**
  - **enter if token is received and own entry is at the top of the queue; delete the entry from the queue**
- **To release critical section**
  - **if queue is non-empty, delete first entry from the queue, send token to that node and make *Holder* variable point to that node**
  - **If queue is still non-empty, send a REQUEST message to the parent (node pointed at by *Holder* variable) {because you want it to come back to serve the pending request}**

Notable features

- **Average message complexity: O(log n)**
- **Sync. delay = (T log n)/2, where T = max. message delay**
- **Starvation ?**

Lets Compare

Algorithm	SyncDelay	Msg
NON TOKEN		
Lamport	N	3(N-1)
Ricart Agrawala	N	2(N-1)
Maekawa	N	3 sqrt(N)
TOKEN BASED		
Suzuki Kasami	N	N
Raymond	(logN)	logN